

# TOY ISA 1

## Instruction Set Architecture Specification

Proteus Lab

Generated on: 2026-09-17

Version: 1.1

## Table of Contents

---

|                       |    |
|-----------------------|----|
| Architecture Overview | 3  |
| BDEP                  | 4  |
| SSAT                  | 5  |
| BEQ                   | 6  |
| CLS                   | 7  |
| NOR                   | 8  |
| LD                    | 9  |
| CBIT                  | 10 |
| J                     | 11 |
| ADDI                  | 12 |
| JALR                  | 13 |
| ST                    | 14 |
| SYSCALL               | 15 |
| ADD                   | 16 |
| STP                   | 17 |
| LI                    | 18 |

## Architecture Overview

### Register file and ABI

|                      |             |             |             |
|----------------------|-------------|-------------|-------------|
| X0<br>return value   | X1<br>arg 0 | X2<br>arg 1 | X3<br>arg 2 |
| X4<br>arg 3          | X5<br>arg 4 | X6<br>arg 5 | X7<br>arg 6 |
| X8<br>syscall number | X9          | X10         | X11         |
| X12                  | X13         | X14         | X15         |
| X16                  | X17         | X18         | X19         |
| X20                  | X21         | X22         | X23         |
| X24                  | X25         | X26         | X27         |
| X28                  | X29         | X30         | X31<br>LR   |

32 registers, 32-bit each

|                       |
|-----------------------|
| PC<br>program counter |
|-----------------------|

### Memory

Memory is byte-addressed and little-endian. A word is 32 bits wide.

## BDEP

### Encoding

| 31:26  | 25:21 | 20:16 | 15:11 | 10:6  | 5:0    |
|--------|-------|-------|-------|-------|--------|
| 000000 | rd    | rs1   | rs2   | 00000 | 011110 |

### Assembler

```
BDEP rd, rs1, rs2
```

### Semantics

```
 $X[rd] \leftarrow \text{bit\_deposit}(X[rs1], X[rs2])$ 
```

### Notes

Places the least significant bits of  $x[rs1]$  into the positions specified by the mask  $x[rs2]$ . The remaining bits are filled with zeros.

## SSAT

### Encoding

| 31:26  | 25:21 | 20:16 | 15:11 | 10:0         |
|--------|-------|-------|-------|--------------|
| 100000 | rd    | rs    | imm5  | 000000000000 |

### Assembler

```
SSAT rd, rs, #imm5
```

### Semantics

```
 $X[rd] \leftarrow \text{saturate\_signed}(X[rs], \text{imm5})$ 
```

### Notes

Saturation of the signed value in  $x[rs]$  to  $N$  bits, where  $N = \#imm$

## BEQ

### Encoding

| 31:26  | 25:21 | 20:16 | 15:0   |
|--------|-------|-------|--------|
| 010010 | rs    | rt    | offset |

### Assembler

```
BEQ rs, rt, #offset
```

### Semantics

```
target ← sign_extend(offset) << 2
```

```
cond ← X[rs] == X[rt]
```

```
PC ← if (cond) PC + target else PC + 4
```

## CLS

### Encoding

| 31:26  | 25:21 | 20:16 | 15:6       | 5:0    |
|--------|-------|-------|------------|--------|
| 000000 | rd    | rs    | 0000000000 | 110111 |

### Assembler

```
CLS rd, rs
```

### Semantics

```
 $X[rd] \leftarrow \text{count\_leading\_signs}(X[rs])$ 
```

### Notes

Counts the number of leading ones bits in the  $X[rs]$  register.

## NOR

### Encoding

| 31:26  | 25:21 | 20:16 | 15:11 | 10:6  | 5:0    |
|--------|-------|-------|-------|-------|--------|
| 000000 | rs    | rt    | rd    | 00000 | 001010 |

### Assembler

```
NOR rd, rs, rt
```

### Semantics

```
 $X[rd] \leftarrow X[rs] \text{ nor } X[rt]$ 
```

## LD

### Encoding

| 31:26  | 25:21 | 20:16 | 15:14 | 13:0 |
|--------|-------|-------|-------|------|
| 000010 | base  | rt    | 00    | imm  |

### Assembler

```
LD rt, [base, #imm]
```

### Semantics

```
addr ← X[base] + sign_extend(imm)
```

```
raise MisalignedAccess unless isAligned(addr)
```

```
X[rt] ← memory[addr]
```

## CBIT

### Encoding

| 31:26  | 25:21 | 20:16 | 15:11 | 10:0       |
|--------|-------|-------|-------|------------|
| 011111 | rd    | rs    | imm5  | 0000000000 |

### Assembler

```
CBIT rd, rs, #imm5
```

### Semantics

```
 $X[rd] \leftarrow \text{clear\_bit\_field}(X[rs], \text{imm5})$ 
```

### Notes

Clears exactly one bit at position #imm in the  $X[rs]$  register to 0. All other bits remain unchanged.

## J

### Encoding

| 31:26  | 25:0        |
|--------|-------------|
| 110010 | instr_index |

### Assembler

J target

### Semantics

$PC \leftarrow (PC \ \& \ 0xF0000000) \mid (instr\_index \ll 2)$

## ADDI

### Encoding

| 31:26  | 25:21 | 20:16 | 15:0 |
|--------|-------|-------|------|
| 111011 | rs    | rt    | imm  |

### Assembler

```
ADDI rt, rs, #imm
```

### Semantics

```
 $X[rt] \leftarrow X[rs] + \text{sign\_extend}(imm)$ 
```

## JALR

### Encoding

| 31:26  | 25:21 | 20:16 | 15:0 |
|--------|-------|-------|------|
| 101001 | rs    | rt    | imm  |

### Assembler

```
JALR rt, rs, #imm
```

### Semantics

```
link ← PC + 4
```

```
PC ← (X[rs] + sign_extend(imm)) & 0xFFFFFFFF
```

```
X[rt] ← link
```

## ST

### Encoding

| 31:26  | 25:21 | 20:16 | 15:14 | 13:0 |
|--------|-------|-------|-------|------|
| 101100 | base  | rt    | 00    | imm  |

### Assembler

```
ST rt, [base, #imm]
```

### Semantics

```
addr ← X[base] + sign_extend(imm)
```

```
raise MisalignedAccess unless isAligned(addr)
```

```
memory[addr] ← X[rt]
```

## SYSCALL

### Encoding

| 31:26  | 25:6 | 5:0    |
|--------|------|--------|
| 000000 | code | 100011 |

### Assembler

```
SYSCALL
```

### Semantics

```
SigException(SystemCall)
```

### Notes

x8 — system call number, x0 - x7 — args, x0 — result, see man syscall

## ADD

### Encoding

| 31:26  | 25:21 | 20:16 | 15:11 | 10:6  | 5:0    |
|--------|-------|-------|-------|-------|--------|
| 000000 | rs    | rt    | rd    | 00000 | 011011 |

### Assembler

```
ADD rd, rs, rt
```

### Semantics

```
 $X[rd] \leftarrow X[rs] + X[rt]$ 
```

## STP

### Encoding

| 31:26  | 25:21 | 20:16 | 15:11 | 10:0   |
|--------|-------|-------|-------|--------|
| 111000 | base  | rt1   | rt2   | offset |

### Assembler

```
STP rt1, rt2, offset(base)
```

### Semantics

```
addr ← X[base] + sign_extend(offset)
```

```
raise MisalignedAccess unless isAligned(addr)
```

```
memory[addr] ← X[rt1]
```

```
memory[addr + 4] ← X[rt2]
```

## LI

### Encoding

| 31:26  | 25:21 | 20:16 | 15:0 |
|--------|-------|-------|------|
| 011001 | 00000 | rt    | imm  |

### Assembler

```
LI rt, #imm
```

### Semantics

```
 $X[rt] \leftarrow \text{sign\_extend}(\text{imm})$ 
```